Contiguous Memory Allocation

READ, RECALL, THEN PRACTISE

StudyNotes original PDF pages **31-38**. Notes begin on page 2 of this pack. Original content and annotations are preserved.

- Check bounds before adding the base address.

- Explain internal/external fragmentation and coalescing/compaction.

- Find holes and apply first-fit, best-fit and worst-fit.

SELECTED PRACTICE

QZ: memory placement / original PDF pages 2, 4

WR: fragmentation explanation / original PDF pages 11

Each practice section has a source page and question guide. Mixed-topic source pages may appear in more than one pack.

Lecture 4 — Memory Concept and Contiguous Memory Allocation

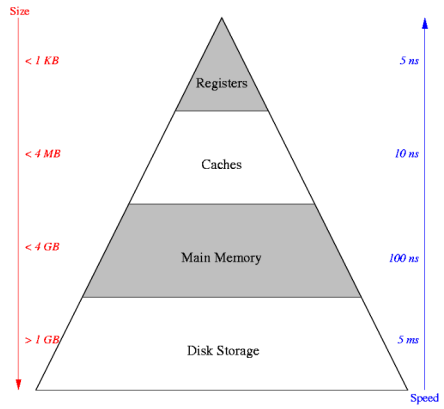
อ้างอิง: Lecture 4 หน้า 1–35 | address translation, Base/Bounds, contiguous allocation และ fragmentation

> **หมายเหตุเรื่องการตรวจสไลด์ Lecture 4**
>
> บทนี้สรุปหัวข้อ Lecture 4 จากภาพสไลด์ที่กู้คืนได้ 7 หน้าและบัญชีหัวข้อเดิม อย่างไรก็ตามไฟล์ PDF ต้นฉบับเสียหาย จึงยังตรวจเทียบรายละเอียดกับสไลด์ครบทั้ง 35 หน้าไม่ได้

4.1 Memory Hierarchy และหน้าที่ของ OS

Lecture เริ่มจากทบทวนลำดับชั้นของหน่วยความจำ เพื่อเปรียบเทียบความจุและความเร็วของแต่ละชนิด

ภาพประกอบ 4.1 Lecture 4 หน้า 3 — Memory Hierarchy Review



จุดสังเกต

- เมื่อพิจารณาจาก register และ cache ไปสู่ main memory และ disk จะพบว่า capacity เพิ่มขึ้น แต่ latency ก็สูงขึ้น
- Lecture 4 เน้น main memory ส่วน Lecture 6 จะกล่าวถึง disk ในฐานะ backing store
- แม้ executable จะเก็บอยู่บน disk แต่ CPU นำคำสั่งมาทำงานจาก disk โดยตรงไม่ได้ ส่วนที่ต้องใช้จึงต้องถูกนำเข้า main memory ก่อน

ที่มา: Lecture 4 หน้า 3.

Register และ cache มีความจุน้อยแต่เข้าถึงได้เร็วกว่า main memory ส่วน disk มีความจุสูงกว่าแต่ใช้เวลาเข้าถึงนานที่สุด โปรแกรมเก็บ executable ไว้บน disk และต้องนำคำสั่งกับข้อมูลที่จำเป็นเข้า main memory ก่อนเริ่มทำงาน

> **เนื้อหาจาก Lecture**
>
> หน้าที่ของ OS ด้าน memory ที่สไลด์หน้า 4 เน้นมีสามแกน:
> 1. ทำ virtualize, manage และ organize memory
> 2. จัด abstraction ให้โปรแกรมใช้งานได้สะดวก โดยคำนึงถึงทั้งเวลาและพื้นที่
> 3. ป้องกันและแยกพื้นที่ของ process ออกจากกัน

4.2 Memory / Address Space Abstraction

Physical Address คือตำแหน่งที่ memory hardware ใช้จริง ส่วน **Logical Address / Virtual Address** คือตำแหน่งที่ program อ้างอิงผ่าน abstraction ของ OS แต่ละ process จึงเห็น address space ของตนเอง แม้ตำแหน่งจริงจะอยู่ใน physical memory ร่วมกัน

> **จุดที่มักสับสน**
>
> **Logical/Virtual Address ต่างจาก Physical Address** address ที่โปรแกรมเห็นไม่จำเป็นต้องตรงกับตำแหน่งจริงใน RAM ระบบจึงต้องแปล address ก่อนเข้าถึง physical memory

4.2.1 Programmable Machine → Time Sharing

ใน programmable machine รุ่นแรก มี program เดียวใช้ memory ส่วนใหญ่ร่วมกับ OS เมื่อระบบพัฒนาเป็น time-sharing และ multiprogramming แต่ละ process จึงต้องมี address space ของตนเอง ทำให้ต้องแก้ปัญหา mapping, protection และ relocation

4.3 Address Translation / Address Binding

Address Translation คือการแปลง logical/virtual address เป็น physical address Lecture แบ่งช่วงเวลาที่ทำ address binding ได้ดังนี้

ช่วงเวลา	แนวคิด	ผลที่ตามมา
Compile time	หากทราบตำแหน่ง memory ล่วงหน้า จะสร้าง absolute code ได้	หากเปลี่ยนตำแหน่งเริ่มต้น ต้อง compile ใหม่
Link/Load time	สร้าง relocatable code แล้วกำหนดตำแหน่งตอน link หรือ load	เปลี่ยนตำแหน่งได้ก่อนเริ่มทำงาน แต่โดยทั่วไปต้อง load ใหม่
Execution time	เลื่อน การ กำหนด ตำแหน่ง ไป จนถึง runtime และย้าย process ได้ระหว่างทำงาน	ต้องมี hardware ช่วยแปล address แบบ dynamic

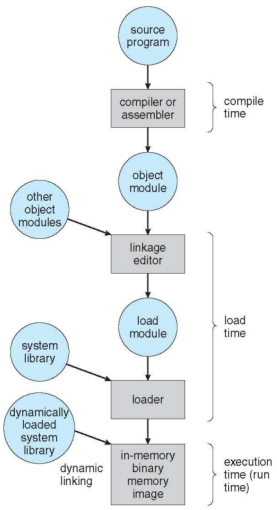
> **เชื่อมกับ Computer Architecture**
>
> Execution-time binding ต้องมี hardware อยู่ในเส้นทางของ memory reference เพราะ CPU สร้าง address ระหว่างทำงาน ระบบจึงต้องแปล address และตรวจสิทธิ์ก่อนเข้าถึง memory กลไกนี้เป็นพื้นฐานของ MMU, Page Table และ TLB ที่จะเรียนใน Lecture 5

ภาพประกอบ 4.2 Lecture 4 หน้า 8 — Address Translation / Binding

แปลง address ขั้นตอนไหน?

Address Translation

Translate logical (virtual) address to physical address (aka. address binding)



8

จุดสังเกต

- Program อ้างอิง logical/virtual address ส่วน memory hardware ใช้ physical address
- Binding ทำได้ตอน compile, load หรือ execution ให้สังเกตว่าตำแหน่งจริงถูกกำหนดเมื่อใด
- หากทำ binding ตอน execution ต้องมี hardware แปล address ในเส้นทางเข้าถึง memory

ที่มา: Lecture 4 หน้า 8.

4.4 Programming API Perspective: code, stack, heap

Lecture แยกมุมมอง storage class ของภาษา C ออกจากชนิดหน่วยความจำที่ OS จัดการ

- **Code**: program instructions
- **Stack**: data ที่ compiler/runtime จัดตำแหน่งให้อัตโนมัติ เช่น local/Auto storage
- **Heap**: dynamic data ที่ programmer ขอผ่าน allocator เช่น `malloc()`

บาง OS อาจแบ่ง segment มากกว่าสามส่วน

4.5 Memory Management and Organization

Lecture แยกคำถามด้านการจัดการ memory ออกเป็นเรื่องการจัดสรรและการเลือกตำแหน่ง

- **Memory Allocation**: single/multiple processes, static/dynamic, contiguous/noncontiguous
- **Placement / Replacement**: ตัดสินว่าข้อมูลใหม่ควรอยู่ตำแหน่งใด และภายหลังใน VM ว่าอะไรควรถูกแทนออก

4.5.1 Contiguous vs Noncontiguous

Contiguous Allocation (การจัดวางต่อเนื่อง): address space ทั้งหมดของ process ต้องอยู่ใน block เดียวของ physical memory หากไม่มี hole ที่ใหญ่พอ ก็ไม่สามารถจัดวาง process ได้

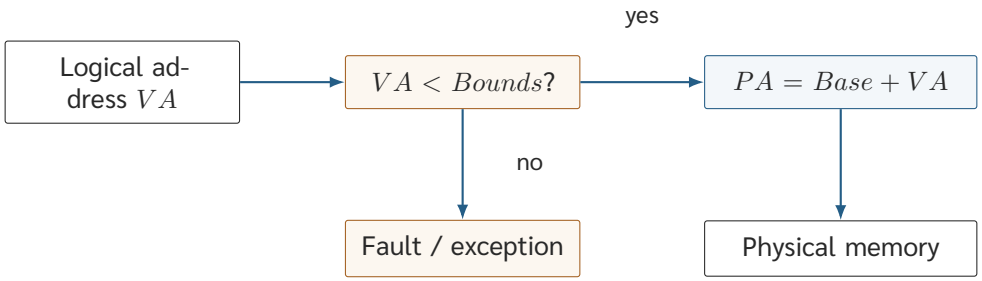
Noncontiguous Allocation (การจัดวางแบบไม่ต่อเนื่อง): แบ่ง process เป็น pages หรือ segments แล้ววางไว้คนละตำแหน่งใน physical memory จึงใช้พื้นที่ว่างขนาดเล็กได้ดีขึ้น Lecture 5 จะอธิบายวิธีนี้ต่อ

4.6 Single Partition Allocation และ Boundary Scheme

สมมติว่ามี process เดียวกำลังทำงาน และ address space เล็กกว่า physical memory **Boundary Scheme** ใช้ register สองตัว:

- **Base / Relocation Register**: physical address เริ่มต้น
- **Bounds / Limit Register**: ขนาด/ช่วง logical address ที่อนุญาต

ตามแบบจำลองใน Lecture CPU มี register คู่นี้หนึ่งชุด



ตัวอย่างคำนวณ

ตัวอย่างใน Lecture กำหนด address space ตั้งแต่ 0 ถึง $4\text{KB} - 1$ โดยมี Base=16 KB=16384 B และ Bounds=4 KB=4096 B
General rule:

$$0 \le VA < Bounds \Rightarrow PA = Base + VA$$

$$\begin{aligned} VA = 0 &\rightarrow PA = 16384 \\ VA = 1024\ \text{B} &\rightarrow 16384 + 1024 = 17408\ \text{B} \\ VA = 3000\ \text{B} &\rightarrow 16384 + 3000 = 19384\ \text{B} \\ VA = 4400\ \text{B} &\rightarrow \text{out of bounds} \rightarrow \text{Fault} \end{aligned}$$

จุดเน้นก่อนสอบ

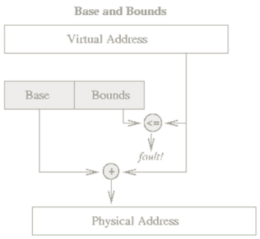
ทำ address problem ให้เรียงสองขั้นเสมอ: **(1) bounds check ก่อน (2) ค่อย add Base.** การบวกก่อนแล้วค่อยดูทีหลังทำให้หลุด protection logic ของ scheme.

ภาพประกอบ 4.3 Lecture 4 หน้า 18 — Base / Bounds Worked Example

Example

- Suppose the address space of the process is 0 to 4KB-1
- Suppose the Base register contains 16KB and the Bounds register contains 4KB
- The virtual address is compared with the Bounds register contents to check within bounds
- The virtual address is added to the Base register contents to give the physical address

Virtual	→	Physical Address
0	→	16 KB (16384)
1 KB	→	17 KB (17408)
3000	→	19384
4400	→	*Fault (out of bounds)*



18

จุดสังเกต

- Logical address ต้องผ่าน **bounds check** ก่อน
- valid แล้วค่อยคำนวณ $PA = Base + VA$
- ตัวอย่าง Base=16KB, Bounds=4KB ทำให้เห็นทั้ง relocation และ protection ใน register pair เดียว

ที่มา: Lecture 4 หน้า 18.

4.7 Multiple Partition Allocation

เมื่อรองรับหลาย process physical memory ถูกแบ่ง partition. Lecture แบ่งสองแบบใหญ่:

- **Fixed-size partitions**: partition ขนาดคงที่ เปลี่ยน runtime ไม่ได้
- **Flexible/Variable/Dynamic partitions**: partition ขนาดตาม process

ทั้งสองยังเป็น contiguous allocation: address space ของ process หนึ่งยังต้องอยู่ติดกันใน physical memory

4.8 Static vs Dynamic Relocation

4.8.1 Static Relocation

Loader rewrite address ใน executable ด้วย relocation value ก่อน run. Lecture example instruction อ้าง logical address 1000 และ relocation value 3000 จึง loader เปลี่ยนเป็น address 4000.
ข้อจำกัดตาม Lecture:

- protection ไม่แข็งแรง — process อาจสร้าง bad address
- relocation value เปลี่ยนไม่สะดวก; ต้อง reload

4.8.2 Dynamic Relocation

Base/Bounds ถูกใช้ระหว่าง runtime; แต่ละ process มี relocation value ของตัวเองและ OS เปลี่ยน Base/Bounds เมื่อ context switch

> **เชื่อมกับ Computer Architecture**
>
> Lecture ระบุ hardware requirements ชัด: ต้องมี Base/Bounds registers, circuitry สำหรับ translate + bounds check, privileged instructions เพื่อ update registers, ability to raise exception เมื่อ illegal access, และ privileged mechanism สำหรับ register exception handlers. เหตุผลที่ instruction แก้ Base/Bounds ต้อง privileged: ถ้า user process ตั้ง Base เองได้ มันสามารถ map ไป physical memory ของ process/kernel อื่นได้

ภาพประกอบ 4.4 Lecture 4 หน้า 32 — Hardware Requirements for Base/Bounds

Hardware Requirements

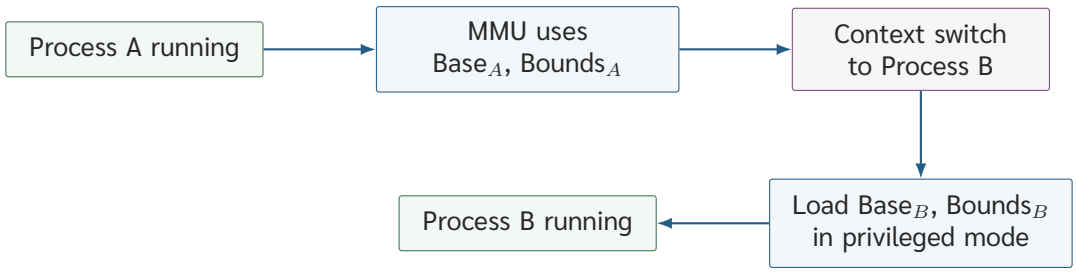
- Base/Bounds pair of registers
  - *Need pair per CPU to support address translation and bounds checks*
- Ability to translate virtual addresses and check if within limits
  - *Circuitry to do translations and check bounds limits*
- Privileged instruction(s) to update Base/Bounds registers
  - *Must be able to set these values before letting a program run*
- Ability to raise exceptions
  - *When processes try to access privileged instructions or out-of-bounds memory*
- Privileged instruction(s) to register exception handlers
  - *OS must be able to tell hardware what code to run if exception occurs*

32

จุดสังเกต

- ต้องมี register pair + circuitry แปล address และ bounds check ใน memory-reference path
- การตั้ง Base/Bounds และ exception handler ต้องเป็น **privileged operation**
- illegal/out-of-bounds access ต้องทำให้ hardware raise exception เข้าสู่ OS

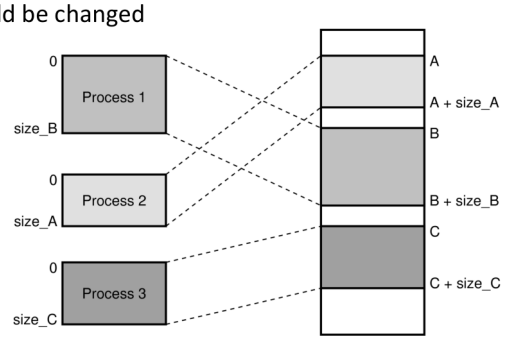
ที่มา: Lecture 4 หน้า 32 ภาพนี้เชื่อม memory virtualization กับ hardware support



ภาพประกอบ 4.5 Lecture 4 หน้า 26 — Dynamic Relocation + Context Switch

Example of Dynamic Relocation

- After context switching, the Base and Bounds register pair should be changed



26

จุดสังเกต

- แต่ละ process มี Base/Bounds ของตัวเอง
- ตอน context switch kernel ต้องโหลด pair ของ process ใหม่ให้ CPU/MMU ใช้
- นี่คือ connection โดยตรงระหว่าง Process context (Lecture 2) กับ Memory protection (Lecture 4)

ที่มา: Lecture 4 หน้า 26.

4.9 Fixed-Size Partitions: Internal Fragmentation

ถ้า executable/process เล็กกว่า partition พื้นที่ที่เหลือใน partition ถูกจองไว้แต่ใช้ไม่ได้ จึง **Internal Fragmentation**. ถ้า process ใหญ่กว่า partition ก็ใส่ไม่ได้เลย

อธิบายให้เห็นภาพ

คำว่า internal ช่วยจำตำแหน่งของ waste: พื้นที่สูญเปล่าอยู่ **ภายใน** block/partition ที่ถูก allocate ให้ process แล้ว

ภาพประกอบ 4.6 Lecture 4 หน้า 27 — Fixed Partitions → Internal Fragmentation

Problem of Fixed-sized Partitions

- An executable program is too small for a partition
  - That mean wasting memory space as some of the partition will be unused – this is called *internal fragmentation* ของเกินแต่ไม่ใช้
- An executable program is too big to fit a partition

27

จุดสังเกต

- process เล็กกว่า partition จึง พื้นที่ **ภายใน partition ที่จัดให้แล้ว** ถูกทิ้ง = internal fragmentation
- อีกกรณีคือ program ใหญ่กว่า partition จนใส่ไม่ได้ แม้ total memory อาจยังเหลือ
- จำ keyword จากตำแหน่งของ waste: **inside allocated region** = internal

ที่มา: Lecture 4 หน้า 27 หมายเหตุในสไลด์ชี้ว่าพื้นที่ที่จองไว้ไม่ได้ถูกใช้

4.10 Flexible-Size Partitions: External Fragmentation

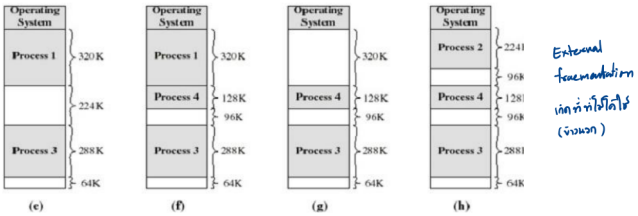
Flexible partitions จัดพื้นที่ตามขนาด process จึงลด internal fragmentation แต่เมื่อ process terminate จะทิ้ง holes หลายก้อน ถ้าพื้นที่ว่างรวมพอแต่ไม่มี hole เดียวใหญ่พอสำหรับ process ใหม่ จะเกิด **External Fragmentation**.

Lecture ใช้ตัวอย่าง physical memory 1024KB, OS 128KB และ job sizes 320,224,288,128,224,192KB เพื่อแสดงการวางและเกิด holes เมื่อ process เข้า/ออก

ภาพประกอบ 4.7 Lecture 4 หน้า 29 — External Fragmentation

External Fragmentation

- External Fragmentation occurs when processes are terminated, and no hole spaces are big enough for a new incoming process (suppose we want to execute a process which is 192KB)
- (1)320KB, (2)224KB, (3)288KB, (4)128KB, (5)224KB, (6)192KB



29

จุดสังเกต

- Free memory รวมอาจพอ แต่แตกเป็น holes ที่ไม่มี block เดียวใหญ่พอ
- นี่คือ **external** fragmentation: waste อยู่ระหว่าง allocations
- Coalescing รวม hole ที่ติดกัน; Compaction ย้าย allocations เพื่อรวม free space แต่มี cost สูงกว่า

ที่มา: Lecture 4 หน้า 29.

4.10.1 Coalescing vs Compaction

วิธีการ	ทำอะไร	ข้อสังเกต
Coalescing	รวม free space ที่อยู่ติดกันให้เป็น hole ที่ใหญ่ขึ้น	ไม่ต้องย้าย block ที่ยังถูกใช้งาน
Compaction	ย้าย process เพื่อรวม free space ให้เป็นก้อนใหญ่	ต้อง อาศัย dynamic relocation เพราะ ต้องเปลี่ยน Base หลังย้าย

> **จุดที่มักสับสน**
>
> **Internal ต่างจาก External Fragmentation** แบบ fixed partition ทำให้พื้นที่ว่างเหลืออยู่ภายใน partition ที่จัดให้ process แล้ว ส่วนแบบ flexible partition ทำให้เกิด holes คั่นระหว่างพื้นที่ที่ถูกจัดสรร วิธีแก้จึงต่างกัน

4.11 Free List และ lifecycle ของ memory

เมื่อสร้าง process OS จะค้น data structure เช่น **free list** เพื่อหาพื้นที่สำหรับ address space แล้วทำเครื่องหมายว่าถูกใช้งาน เมื่อ process จบ OS คืน memory เข้า free list และล้าง structures ที่เกี่ยวข้อง นอกจากนี้ Base/Bounds ต้องถูกบันทึกและคืนค่าระหว่าง context switch ส่วน dynamic relocation ทำให้ย้าย process ที่หยุดอยู่ไปยังตำแหน่งอื่นใน memory ได้

4.12 Trailing Lecture Material: C Storage Classes

Lecture หน้า 34–35 กลับมาทบทวนมุมมอง programming API:

- **Auto**: default local variables; lifetime ตาม function/block
- **Static**: local variable คงอยู่ตลอด program lifetime
- **Extern**: reference global variable ที่เห็นข้าม program files
- **Register**: ขอให้ local variable อยู่ register แทน RAM ตาม framing ของ Lecture

ตัวอย่าง Auto: variable จะเริ่มมีอายุเมื่อเข้า function หรือ block และสิ้นสุดเมื่อออกจากบริเวณนั้น ดังนั้น block-local variable เช่น k จึงใช้นอก block ไม่ได้

> **จุดที่มักสับสน**
>
> ส่วน C Storage Classes อยู่ใน Lecture จริง แต่เป็น programming-API perspective และไม่ควรเอาไปปนกับ OS-level Base/Bounds หรือ segmentation scheme. ใช้เพื่อเชื่อมว่าภาษา/compile-time view กับ OS memory view เป็นคนละ abstraction level.

4.13 Chapter Summary

> **สรุปท้ายบท**
>
> - OS virtualizes main memory เพื่อ transparency, efficiency, protection/isolation
> - Logical address ต้องถูก bind/translate เป็น physical address; execution-time binding ต้อง hardware support
> - Base/Bounds ทำ translation แบบง่าย: check Bounds แล้ว add Base
> - Dynamic relocation เปลี่ยน Base/Bounds ตาม process และรองรับ moving process; update ต้อง privileged
> - Fixed partitions จึง internal fragmentation; Flexible partitions จึง external fragmentation
> - Coalescing รวม holes ที่ติดกัน; Compaction ย้าย blocks เพื่อรวม free space และต้อง dynamic relocation
> - Contiguous allocation มีข้อจำกัดที่นำไปสู่ Noncontiguous Allocation ใน Lecture 5

Key Mechanisms

Address translation, Base/Bounds check, exception on invalid access, save/restore Base/Bounds on context switch.

Key Policies

Contiguous placement choice, fixed vs flexible partitioning, free-list management, coalescing/compaction.

Formula / Calculation Recap

$$VA < Bounds \Rightarrow PA = Base + VA; \qquad VA \geq Bounds \Rightarrow Fault$$

หน่วย Base, Bounds, VA ต้องอยู่ unit เดียวกันก่อนบวก/เปรียบเทียบ

ComArch Connections

Physical main memory, registers, privileged register update, compare/add circuitry, exception handler path.

Common Confusions

- logical address ต่างจาก physical address
- Base = starting physical address; Bounds = legal range/size ไม่ใช่ ending address โดยตรงใน lecture model
- Static relocation rewrite address ก่อน run; Dynamic relocation ทำ translation runtime
- internal fragmentation ต่างจาก external fragmentation
- Context switch เปลี่ยน Base/Bounds ของ current process แต่ไม่เท่ากับ moving process ใน memory เสมอไป

Quick Revision Checklist

☐ แยก compile/load/execution-time binding ได้
☐ ทำ Base/Bounds translation และ detect fault ได้
☐ อธิบายเหตุผลที่ Base/Bounds update ต้อง privileged ได้
☐ วาด fixed vs flexible partitions และชี้ internal/external fragmentation ได้
☐ อธิบาย coalescing vs compaction ได้

Practice: QZ: memory placement

ORIGINAL SOURCE

Quiz_BeforeMidterm.pdf

Original PDF page(s): **2, 4**

The two pages use different process sizes. Rebuild the memory map for each page.

ANSWER ANNOTATIONS

Handwritten, highlighted and pre-selected answers are part of the source and are not certified solutions. Hide them while practising. Recomputed examples and explicit assumptions are in CHECKED_DRILLS and the main study guide.

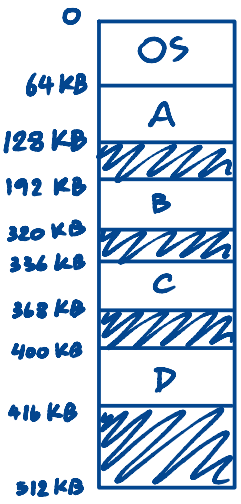
2. A computer system has a total memory of 512 KB. It supports multiprogramming with contiguous memory allocation using flexible-size partitions. The operating system uses 64 KB memory, starting at address 0. At a given moment in time, the memory is occupied by four processes, A, B, C and D, as shown in table below

Process	Base Address	Size
A	64 KB	64 KB
B	192 KB	128 KB
C	336 KB	32 KB
D	400 KB	16 KB

a) List the areas of free memory (holes) giving their start address and size.

b) Suppose the operating system receives a request from a new process E for 30 KB memory to be allocated. Assuming the free memory is searched in order of start address, what would be the base address of this new process using each of the following strategies: (i) First-Fit, (ii) Best-Fit, and (iii) Worst-Fit.

a)



	start @	size
Hole 1	128 KB	64 KB
Hole 2	320 KB	16 KB
Hole 3	368 KB	32 KB
Hole 4	416 KB	96 KB

b) First-Fit : เลือก Hole แรกที่ใช้ได้

Base address is 128 KB (Hole 1)

Best-Fit : เลือก Hole ที่ใส่แล้วทำให้พื้นที่เหลือน้อยสุด

Base address is 368 KB (Hole 3)

Worst-Fit : เลือก Hole ที่ใส่แล้วทำให้พื้นที่เหลือมากสุด

Base address is 416 KB (Hole 4)

Question 4: (10 points)

A computer system has a total memory of 512 KB. It supports multiprogramming with contiguous memory allocation using flexible-size partitions. The operating system uses 64 KB memory, starting at address 0. At a given moment in time, the memory is occupied by four processes, A, B, C and D, as shown in Table Q4.

Process	Base Address	Size
A	64 KB	80 KB
B	192 KB	64 KB
C	336 KB	32 KB
D	400 KB	96 KB

Table Q4

(a) **(4 points)** List the areas of free memory (holes) giving their start address and size.

ANS:



hole
↳ 144k → 192k (size = 48kB)
↳ 256k → 336k (size = 80kB)
↳ 368k → 400k (size = 32kB)
↳ 496k → 512k-1 (size = 16kB)

(b) **(6 points)** Suppose the operating system receives a request from a new process for 30 KB memory to be allocated. Assuming the free memory is searched in order of start address, what would be the base address of this new process using each of the following strategies (i) First-Fit, (ii) Best-Fit, and (iii) Worst-Fit. In each case, give a brief explanation of your answer **(2 points each, 1 for answer, 1 for explanation)**.

ANS:

First-Fit: เลือก hole แรกที่เข้าได้ → base: 144kB

Best-Fit: เลือก hole ที่เข้าแล้วเหลือพื้นที่น้อยที่สุด → base: 368k

Worst-Fit: เลือก hole ที่เข้าแล้วเหลือพื้นที่มากที่สุด → base = 256k

Practice: WR: fragmentation explanation

ORIGINAL SOURCE

WR / Written question collection (17 pages)

Full source filename is listed under this code in PRACTICE_INDEX.md.

Original PDF page(s): **11**

Draw both fragmentation cases and check the explanations against Lecture 4.

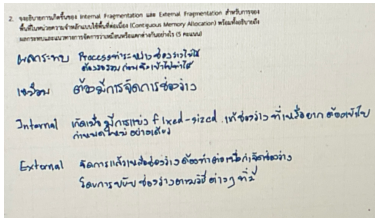
ANSWER ANNOTATIONS

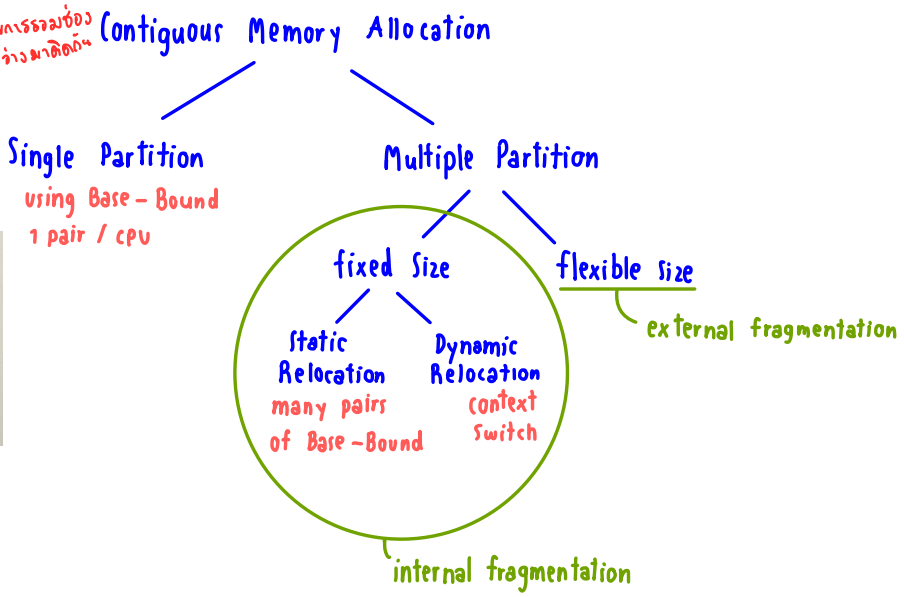
Handwritten, highlighted and pre-selected answers are part of the source and are not certified solutions. Hide them while practising. Recomputed examples and explicit assumptions are in CHECKED_DRILLS and the main study guide.

2. จงอธิบายการเกิดขึ้นของ Internal Fragmentation และ External Fragmentation สำหรับการจองพื้นที่ในหน่วยความจำหลักแบบใช้พื้นที่ต่อเนื่อง (Contiguous Memory Allocation) พร้อมทั้งอธิบายถึงผลกระทบและแนวทางการจัดการว่าเหมือนหรือแตกต่างกันอย่างไร (5 คะแนน)

External จัดการแล้วเหลือช่องว่าง ต้องทำต่อเนื่องถึงจัดช่องว่างโดยการรวมช่องว่างมาติดกัน

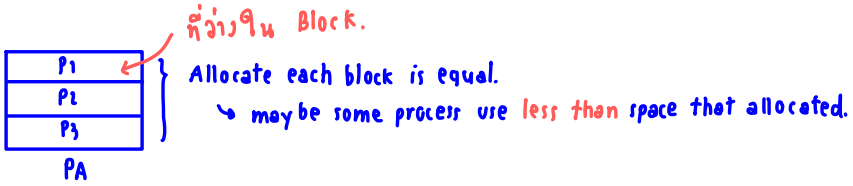
Internal เกิดเมื่อมีการแบ่ง fixed-sized แก้ช่องว่างที่เหลือต่อ segment ได้ยาก ต้องเข้าไปทำใหม่





1. เกิด Internal fragmentation ตอนที่มีการ Allocate แบบ fixed size เพราะ: Program execute use space smaller than the space allocated. Because it not depend on the size of process.

=) Allocate too big space.



2. External fragmentation เกิดขึ้นเมื่อมีการ Allocate แบบตาม size ของแต่ละ Process เพื่อไม่ให้เกิด Internal fragmentation แล้วส่งผลให้เกิดพื้นที่ว่าง (Hole) ใน PA แบบ Non-Contiguous ทำให้ New Process ไม่สามารถ Allocate ได้ เพราะ: เป็นการ Allocate แบบ Contiguous Memory

=) Have total space enough, but it's not contiguous

